Introduction to R Workshop

Getting Started, Data Visualization, and Regression

Muji Chughtai

2026-07-20

Getting Started

Introduction to R and RStudio

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.
  - To download R, go to https://cran.r-project.org.

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.
  - To download R, go to https://cran.r-project.org.
- For a more user-friendly interface, RStudio is a popular integrated development environment (IDE) for R.

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.
  - To download R, go to https://cran.r-project.org.
- For a more user-friendly interface, RStudio is a popular integrated development environment (IDE) for R.
- RStudio also allows for powerful document creation using Markdown, which can generate reports, presentations, and dashboards (this presentation was made using a Quarto Markdown in RStudio).

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.
  - To download R, go to https://cran.r-project.org.
- For a more user-friendly interface, RStudio is a popular integrated development environment (IDE) for R.
- RStudio also allows for powerful document creation using Markdown, which can generate reports, presentations, and dashboards (this presentation was made using a Quarto Markdown in RStudio).
  - To download RStudio, go to https://www.rstudio.com/products/rstudio/download.

Getting Familiar with the RStudio Interface

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

1. **Source Panel (Top Left)**: This is where you can write and edit your R scripts and markdown files. Toggling between tabs allows you to manage multiple files at once.

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

1. **Source Panel (Top Left)**: This is where you can write and edit your R scripts and markdown files. Toggling between tabs allows you to manage multiple files at once.
2. **Console Panel (Bottom Left)**: This is where you can directly type and execute R commands. When you run code from the Source panel, the output will also appear here.

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

1. **Source Panel (Top Left)**: This is where you can write and edit your R scripts and markdown files. Toggling between tabs allows you to manage multiple files at once.
2. **Console Panel (Bottom Left)**: This is where you can directly type and execute R commands. When you run code from the Source panel, the output will also appear here.
3. **Environment/History Panel (Top Right)**: The Environment tab shows the objects (variables, data frames, functions, etc.) currently in your R session.

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

1. **Source Panel (Top Left)**: This is where you can write and edit your R scripts and markdown files. Toggling between tabs allows you to manage multiple files at once.
2. **Console Panel (Bottom Left)**: This is where you can directly type and execute R commands. When you run code from the Source panel, the output will also appear here.
3. **Environment/History Panel (Top Right)**: The Environment tab shows the objects (variables, data frames, functions, etc.) currently in your R session.
4. **Files/Plots/Packages/Help/Viewer Panel (Bottom Right)**: This panel has multiple tabs for managing files, viewing plots, installing and loading packages, accessing R help documentation, and displaying web content.

Using Markdown

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.
- In R Markdown, you write code in **chunks** that are enclosed by triple backticks and curly braces.

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.
- In R Markdown, you write code in **chunks** that are enclosed by triple backticks and curly braces.
  - You can create R chunks using the green "c" button in the RStudio toolbar or by typing in three backticks followed by `{r}`.

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.
- In R Markdown, you write code in **chunks** that are enclosed by triple backticks and curly braces.
  - You can create R chunks using the green "c" button in the RStudio toolbar or by typing in three backticks followed by `{r}`.
  - If you want to write in other languages in the code chunk, you can specify the language in the curly braces (e.g., `{python}`, `{bash}`, etc.).

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.
- In R Markdown, you write code in **chunks** that are enclosed by triple backticks and curly braces.
  - You can create R chunks using the green "c" button in the RStudio toolbar or by typing in three backticks followed by `{r}`.
  - If you want to write in other languages in the code chunk, you can specify the language in the curly braces (e.g., `{python}`, `{bash}`, etc.).
- I suggest you do your analyses in R Markdown documents, as it allows you to keep your code and results together in a reproducible format that can be easily exported to various formats (HTML, PDF, Word, etc.)!

Staying Organized

Staying Organized

- One of the biggest issues that people face when programming (in R) is keeping track of code, files, and output.

Staying Organized

- One of the biggest issues that people face when programming (in R) is keeping track of code, files, and output.
- To help with this, I encourage you to create a dedicate `RProject` for each project you work on. This will keep all of your files in one folder, and within this folder you can have subfolders for `Data`, `Analyses`, `Figures`, etc.

Staying Organized

- One of the biggest issues that people face when programming (in R) is keeping track of code, files, and output.
- To help with this, I encourage you to create a dedicate `RProject` for each project you work on. This will keep all of your files in one folder, and within this folder you can have subfolders for `Data`, `Analyses`, `Figures`, etc.
- The `here` package is a great tool to help you manage file paths in your R projects. It allows you to use relative paths instead of absolute paths, which makes your code more portable and easier to share.

Data Visualization

Creating a Directory

Creating a Directory

Let's start programming in R by creating a small project where we will visualize and analyze some data from using data from the World Bank in 2016.

Creating a Directory

Let's start programming in R by creating a small project where we will visualize and analyze some data from using data from the World Bank in 2016.

- Open RStudio and create a new project (and corresponding folder) by navigating from the menu to `File > New Project > New Directory > New Project`, and name your project whatever you like and save it in a location you prefer.

Creating a Directory

Let's start programming in R by creating a small project where we will visualize and analyze some data from using data from the World Bank in 2016.

- Open RStudio and create a new project (and corresponding folder) by navigating from the menu to `File > New Project > New Directory > New Project`, and name your project whatever you like and save it in a location you prefer.
- Within that project folder, create a new R Markdown document by navigating to `File > New File > R Markdown`, and you can do this from either the menu or the bottom right panel. Name it whatever you like!

Installing and Loading Packages

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: `tidyverse` and `here`.

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: `tidyverse` and `here`.
  - `Tidyverse` is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: `tidyverse` and `here`.
  - `Tidyverse` is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.
  - `Here` is a package that helps you manage file paths in your R projects.

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: `tidyverse` and `here`.
  - `Tidyverse` is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.
  - `Here` is a package that helps you manage file paths in your R projects.
- To load in packages in R you use the `library()` function:

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: `tidyverse` and `here`.
  - `Tidyverse` is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.
  - `Here` is a package that helps you manage file paths in your R projects.
- To load in packages in R you use the `library()` function:

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: `tidyverse` and `here`.
  - `Tidyverse` is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.
  - `Here` is a package that helps you manage file paths in your R projects.
- To load in packages in R you use the `library()` function:

```
library(tidyverse)
library(here)
```

Installing and Loading Packages

- This code wont work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

Installing and Loading Packages

- This code wont work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

Installing and Loading Packages

- This code wont work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

```
install.packages("tidyverse", repos = "https://cloud.r-project.org/")
install.packages("here", repos = "https://cloud.r-project.org/")
```

Installing and Loading Packages

- This code wont work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

```
install.packages("tidyverse", repos = "https://cloud.r-project.org/")
install.packages("here", repos = "https://cloud.r-project.org/")
```

- Newer versions of RStudio will also allow you to download packages from the top menu if you don't have them.

Installing and Loading Packages

- This code wont work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

```
install.packages("tidyverse", repos = "https://cloud.r-project.org/")
install.packages("here", repos = "https://cloud.r-project.org/")
```

- Newer versions of RStudio will also allow you to download packages from the top menu if you don't have them.
- You can also directly install packages from the console by typing in the same code above (when you run a chunk it will run in the console).

Installing and Loading Packages

Installing and Loading Packages

- You can make comments in your code using the `#` symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

Installing and Loading Packages

- You can make comments in your code using the `#` symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

Installing and Loading Packages

- You can make comments in your code using the # symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

```
# loading in required packages
library(tidyverse)
library(here)
```

Installing and Loading Packages

- You can make comments in your code using the # symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

```
# loading in required packages
library(tidyverse)
library(here)
```

- If you use the # symbol outside of a chunk, it will be treated as a header in your R Markdown document. The more # symbols you use, the smaller the header will be.

Installing and Loading Packages

- You can make comments in your code using the # symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

```
# loading in required packages
library(tidyverse)
library(here)
```

- If you use the # symbol outside of a chunk, it will be treated as a header in your R Markdown document. The more # symbols you use, the smaller the header will be.
- These headings will transfer into the output document that you render for easy navigation in the html, pdf, or word document.

Loading in the Data

Loading in the Data

- Let's load in the data from this project, which is a csv file that contains data from the World Bank in 2016.

Loading in the Data

- Let's load in the data from this project, which is a csv file that contains data from the World Bank in 2016.
- The data and codebook is available on my github at https://github.com/mujtabachughtai/2026-R-Workshop.

Loading in the Data

- Let's load in the data from this project, which is a csv file that contains data from the World Bank in 2016.
- The data and codebook is available on my github at https://github.com/mujtabachughtai/2026-R-Workshop.
- Download the data (called `WB.2016.csv`), create a `Data` subfolder in this project's directory, and save the data in that folder.

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

```
WorldBank <- read.csv(
  here("Data", "WB.2016.csv"),
  fileEncoding = "Windows-1252"
)
```

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

```
WorldBank <- read.csv(
  here("Data", "WB.2016.csv"),
  fileEncoding = "Windows-1252"
)
```

- Note: You normally don't need the `fileEncoding` argument, but this dataset has some special characters that require it.

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

```
WorldBank <- read.csv(
  here("Data", "WB.2016.csv"),
  fileEncoding = "Windows-1252"
)
```

- Note: You normally don't need the `fileEncoding` argument, but this dataset has some special characters that require it.
- You should now be able to view your data in the `Environment` tab in the top right panel of RStudio.

Getting a Feel for the Data

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
  - `Country`: The name of the country

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
  - `Country`: The name of the country
  - `Code`: A 3-letter code for each country

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
  - `Country`: The name of the country
  - `Code`: A 3-letter code for each country
  - `Population`: The population of the country

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
  - `Country`: The name of the country
  - `Code`: A 3-letter code for each country
  - `Population`: The population of the country
  - `GNI`: Gross National Income per capita

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
  - `Country`: The name of the country
  - `Code`: A 3-letter code for each country
  - `Population`: The population of the country
  - `GNI`: Gross National Income per capita
- And we're going to make some variables of our own, too.

Getting a Feel for the Data

- Let's take a look at the first few rows of these key variables. We will use the `knitr::kable` and `head` functions to do so.

```
knitr::kable(head(
  WorldBank[, c("Country", "Code", "Population", "GNI")]),
  caption = "First 5 Rows of our Key Variables from the
2016 World Bank Data")
```

Getting a Feel for the Data

Table 1: First 5 Rows of our Key Variables from the 2016 World Bank Data

Country	Code	Population	GNI
Afghanistan	AFG	34656032	580
Albania	ALB	2876101	4320
Algeria	DZA	40606052	4360
American Samoa	ASM	55599	NA
Andorra	AND	77281	NA
Angola	AGO	28813463	3450

Getting a Feel for the Data

Getting a Feel for the Data

- It looks like there is some misingness in data, which is something we should keep in mind as we are doing our analyses.

Getting a Feel for the Data

- It looks like there is some misingness in data, which is something we should keep in mind as we are doing our analyses.
- We can check the means of our key variables using the R's build in mean function:

Getting a Feel for the Data

- It looks like there is some misingness in data, which is something we should keep in mind as we are doing our analyses.
- We can check the means of our key variables using the R's build in mean function:

Getting a Feel for the Data

- It looks like there is some misingness in data, which is something we should keep in mind as we are doing our analyses.
- We can check the means of our key variables using the R's build in mean function:

```
# mean of Population
mean(WorldBank$Population, na.rm = TRUE)
```

```
[1] 34331920
```

Getting a Feel for the Data

- It looks like there is some misingness in data, which is something we should keep in mind as we are doing our analyses.
- We can check the means of our key variables using the R's build in mean function:

```
# mean of Population
mean(WorldBank$Population, na.rm = TRUE)
```

```
[1] 34331920
```

```
# mean of GNI
mean(WorldBank$GNI, na.rm = TRUE)
```

```
[1] 12794.73
```

Getting a Feel for the Data

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

```
WorldBank %>%
  ggplot(aes(x = Population)) +
  geom_density()
```

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

```
WorldBank %>%
  ggplot(aes(x = Population)) +
  geom_density()
```

- This can equivalently be written as

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

```
WorldBank %>%
  ggplot(aes(x = Population)) +
  geom_density()
```

- This can equivalently be written as

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

```
WorldBank %>%
  ggplot(aes(x = Population)) +
  geom_density()
```

- This can equivalently be written as

```
ggplot(aes(x = Population), data = WorldBank) +
  geom_density()
```

Getting a Feel for the Data

- `ggplot` objects add layers to the plot using the `+` operator, and you can add as many layers as you want.

Getting a Feel for the Data

- `ggplot` objects add layers to the plot using the `+` operator, and you can add as many layers as you want.
- Let's make this plot cleaner by changing the theme and additing title and axis labels:

Getting a Feel for the Data

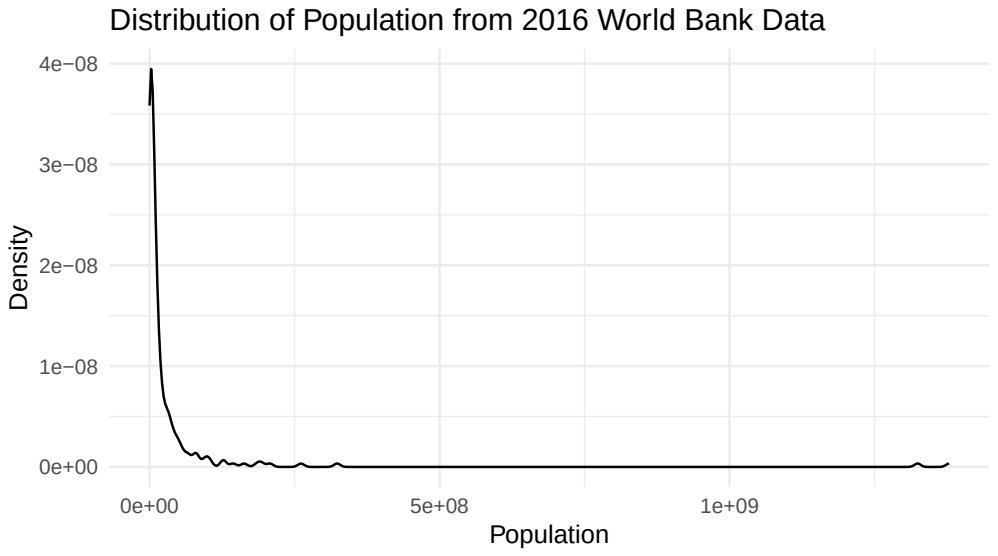
- `ggplot` objects add layers to the plot using the `+` operator, and you can add as many layers as you want.
- Let's make this plot cleaner by changing the theme and additing title and axis labels:

Getting a Feel for the Data

- `ggplot` objects add layers to the plot using the `+` operator, and you can add as many layers as you want.
- Let's make this plot cleaner by changing the theme and additing title and axis labels:

```
WorldBank %>%
  ggplot(aes(x = Population)) +
  geom_density() +
  theme_minimal() +
  labs(
    title = "Distribution of Population from 2016 World Bank Data",
    x = "Population",
    y = "Density"
  )
```

Getting a Feel for the Data



Making Transformations

- The plot looks great, but the data is very heavily right skewed:

Making Transformations

- The plot looks great, but the data is very heavily right skewed:
  - This is because the vast majority of countries that have small populations, while there are a few that have very large ones.

Making Transformations

- The plot looks great, but the data is very heavily right skewed:
  - This is because the vast majority of countries that have small populations, while there are a few that have very large ones.
- We can log-transform the data to make it more normally distributed, which is a common technique in statistics. This will also help us with our regression analyses later on.

Making Transformations

Making Transformations

- The `mutate` function allows you to create new variables based on the data from other variables. Let's use this function to create a new variable called `logPopulation` that is just the natural log of the `Population` variables, and we will recreate the `WorldBank` dataframe with this new variable:

Making Transformations

- The `mutate` function allows you to create new variables based on the data from other variables. Let's use this function to create a new variable called `logPopulation` that is just the natural log of the `Population` variables, and we will recreate the `WorldBank` dataframe with this new variable:

Making Transformations

- The `mutate` function allows you to create new variables based on the data from other variables. Let's use this function to create a new variable called `logPopulation` that is just the natural log of the `Population` variables, and we will recreate the `WorldBank` dataframe with this new variable:

```
WorldBank <- WorldBank %>%
  mutate(logPopulation = log(Population))
```

Making Transformations

Making Transformations

- Now we can recreate the density plot using `logPopulation`:

Making Transformations

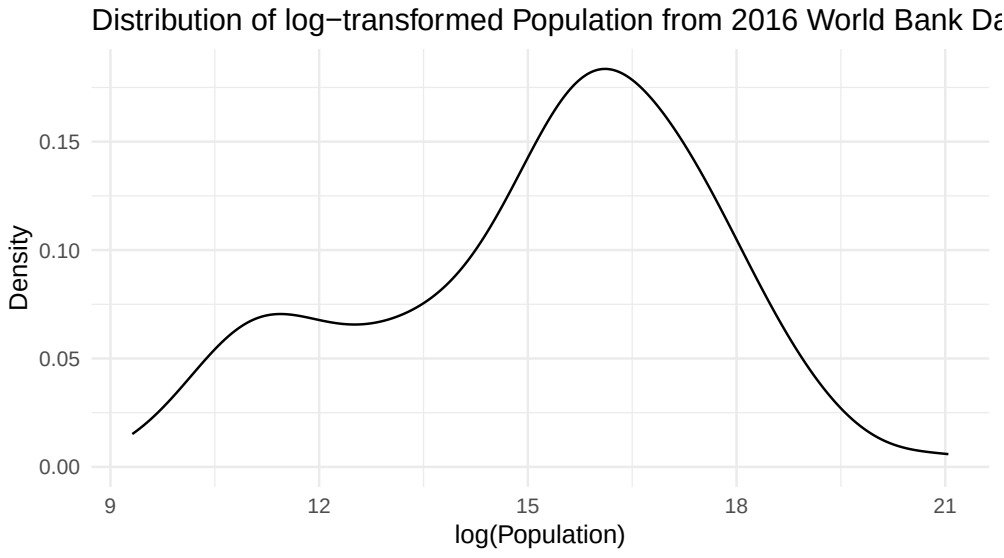
- Now we can recreate the density plot using `logPopulation`:

Making Transformations

- Now we can recreate the density plot using `logPopulation`:

```
WorldBank %>%
  ggplot(aes(x = logPopulation)) +
  geom_density() +
  theme_minimal() +
  labs(
    title = "Distribution of
log-transfored Population from 2016 World Bank Data",
    x = "log(Population)",
    y = "Density"
  )
```

Making Transformations



Making Transformations

Making Transformations

- Let's do the same for GNI

Making Transformations

- Let's do the same for GNI

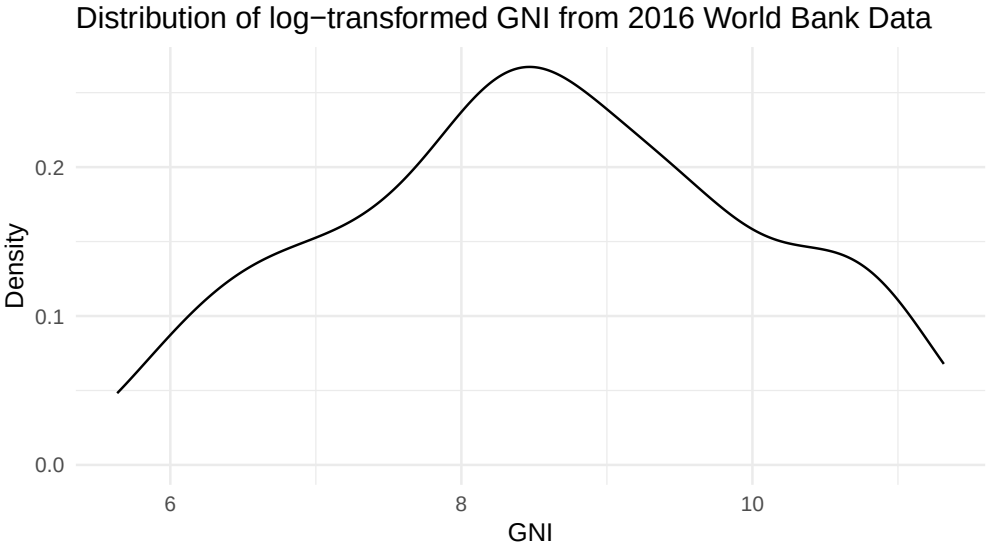
Making Transformations

- Let's do the same for GNI

```
WorldBank <- WorldBank %>%
  mutate(logGNI= log(GNI))

WorldBank %>%
  ggplot(aes(x = logGNI)) +
  geom_density() +
  theme_minimal(base_size = 22) +
  labs(
    title = "Distribution of log-transformed
GNI from 2016 World Bank Data",
    x = "log(GNI)",
    y = "Density"
  )
```

Making Transformations



Plotting Associations

Plotting Associations

- Now we can compare the relationship between a country's GNI and its population by plotting `logPopulation` versus `logGNI`:

Plotting Associations

- Now we can compare the relationship between a country's GNI and its population by plotting `logPopulation` versus `logGNI`:
- We do this by using `geom_point()`.

Plotting Associations

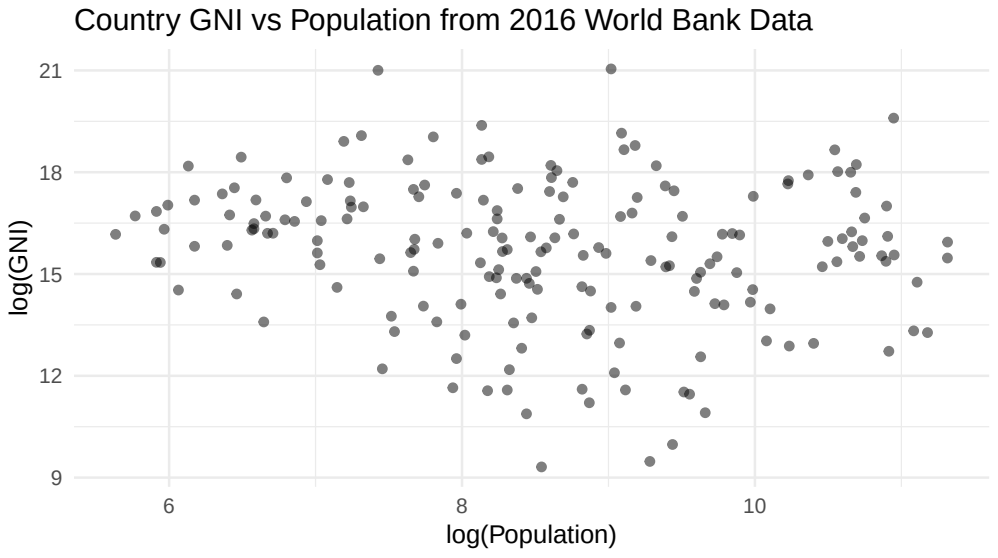
- Now we can compare the relationship between a country's GNI and its population by plotting `logPopulation` versus `logGNI`:
- We do this by using `geom_point()`.

Plotting Associations

- Now we can compare the relationship between a country's GNI and its population by plotting `logPopulation` versus `logGNI`:
- We do this by using `geom_point()`.

```
WorldBank %>%
  ggplot(aes(x = logGNI, y = logPopulation)) +
  geom_point(alpha = 0.5) +
  theme_minimal(base_size = 22) +
  labs(
    title = "Country GNI vs Population from 2016 World Bank Data",
    x = "log(Population)",
    y = "log(GNI)"
  )
```

Plotting Associations



Plotting Associations

Plotting Associations

- We can also add a line of best fit to this plot to see if this relationship tends to be negative or positive (and we will formally test this relationship later using linear regression)

Plotting Associations

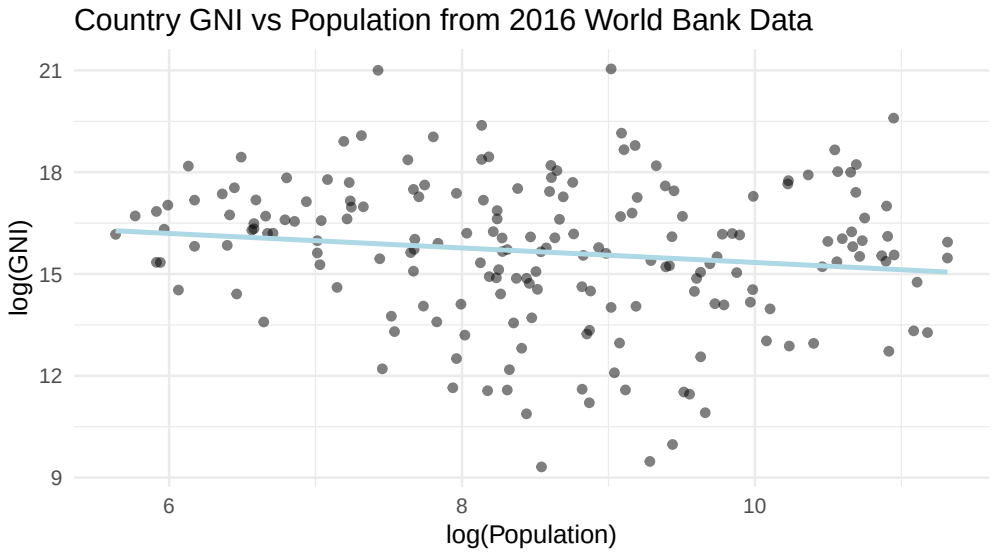
- We can also add a line of best fit to this plot to see if this relationship tends to be negative or positive (and we will formally test this relationship later using linear regression)

Plotting Associations

- We can also add a line of best fit to this plot to see if this relationship tends to be negative or positive (and we will formally test this relationship later using linear regression)

```
WorldBank %>%
  ggplot(aes(x = logGNI, y = logPopulation)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE, color = "lightblue") +
  theme_minimal(base_size = 22) +
  labs(
    title = "Country GNI vs Population from 2016 World Bank Data",
    x = "log(Population)",
    y = "log(GNI)"
  )
```

Plotting Associations



Plotting Associations

Plotting Associations

- It looks like there is a slight negative relationship between a country's GNI and its population, but before we formally test this relationship, we can try to incorporate more information about the underlying country datapoints into the plot.

Plotting Associations

- It looks like there is a slight negative relationship between a country's GNI and its population, but before we formally test this relationship, we can try to incorporate more information about the underlying country datapoints into the plot.
- Specifically, we can color countries based on the continent that they come from to get an initial sense on if the relationship between a country's population and it GNI might differ across different continents.

Plotting Associations

- It looks like there is a slight negative relationship between a country's GNI and its population, but before we formally test this relationship, we can try to incorporate more information about the underlying country datapoints into the plot.
- Specifically, we can color countries based on the continent that they come from to get an initial sense on if the relationship between a country's population and it GNI might differ across different continents.
- However, when looking at the current `WorldBank` dataframe, there is no continent variable.

Plotting Associations

- It looks like there is a slight negative relationship between a country's GNI and its population, but before we formally test this relationship, we can try to incorporate more information about the underlying country datapoints into the plot.
- Specifically, we can color countries based on the continent that they come from to get an initial sense on if the relationship between a country's population and it GNI might differ across different continents.
- However, when looking at the current `WorldBank` dataframe, there is no continent variable.
- We could use `mutate` again to create this new `continent` variable, but there are too many countries to assess the value of to create the corresponding continent.

Combining Dataframes

Combining Dataframes

- Instead, we could take another dataframe (which I have created and is available on my github) that has the continents for each country, and we could combine it to our current `WorldBank` dataframe.

Combining Dataframes

- Instead, we could take another dataframe (which I have created and is available on my github) that has the continents for each country, and we could combine it to our current `WorldBank` dataframe.
- To combine dataframes, you have to have a variable that is shared in both so that rows are combined when the values in that column are shared.

Combining Dataframes

- Instead, we could take another dataframe (which I have created and is available on my github) that has the continents for each country, and we could combine it to our current `WorldBank` dataframe.
- To combine dataframes, you have to have a variable that is shared in both so that rows are combined when the values in that column are shared.
- For us, that would be the `Country` variable

Combining Dataframes

- Instead, we could take another dataframe (which I have created and is available on my github) that has the continents for each country, and we could combine it to our current `WorldBank` dataframe.
- To combine dataframes, you have to have a variable that is shared in both so that rows are combined when the values in that column are shared.
- For us, that would be the `Country` variable
- Let's load in the `CountryContinents.csv` file into our workspace and combine it into a new `WorldBank` dataframe using `left_join`

Combining Dataframes

```
CountryContinents <- read.csv(
  here("Data", "CountryContinents.csv"),
  fileEncoding = "Windows-1252"
)

WorldBank <- WorldBank %>%
  left_join(CountryContinents, by = "Country")
```

Computing Summary Statistics

- When you click on the dataframe, you should now see the `Continent` column.

Computing Summary Statistics

- When you click on the dataframe, you should now see the `Continent` column.
- With this new grouping variable, we can also calculate summary statistics for each group separately. Let's do this before remaking the `logPopulation` vs `logGNI` plot with `Continent`.

Computing Summary Statistics

- When you click on the dataframe, you should now see the `Continent` column.
- With this new grouping variable, we can also calculate summary statistics for each group separately. Let's do this before remaking the `logPopulation` vs `logGNI` plot with `Continent`.
- To calculate summary statistics for each group, we can use the `group_by` and `summarise`:

Computing Summary Statistics

```
WorldBank %>%
  group_by(Continent) %>%
  summarise(
    n = n(),
    mean_logGNI = mean(logGNI, na.rm = TRUE),
    mean_logPopulation = mean(logPopulation, na.rm = TRUE)
  ) %>%
  knitr::kable(caption = "Summary Statistics by Continent",
               col.names = c("Continent", "Number of Countries",
                             "Mean log(GNI)", "Mean log(Population)"))
```

Computing Summary Statistics

Table 2: Summary Statistics by Continent

Continent	Number of Countries	Mean log(GNI)	Mean log(Population)
Africa	54	7.198026	16.03096
Asia	67	8.685735	15.32228
Europe	49	9.803978	14.93849
North/Central America	34	9.045212	13.52896
South America	12	8.842763	16.33919

Plotting 3 Variables

Plotting 3 Variables

- Looks great! Now we can remake the `logPopulation` vs `logGNI` plot and color the points (and lines) by `Continent`:

Plotting 3 Variables

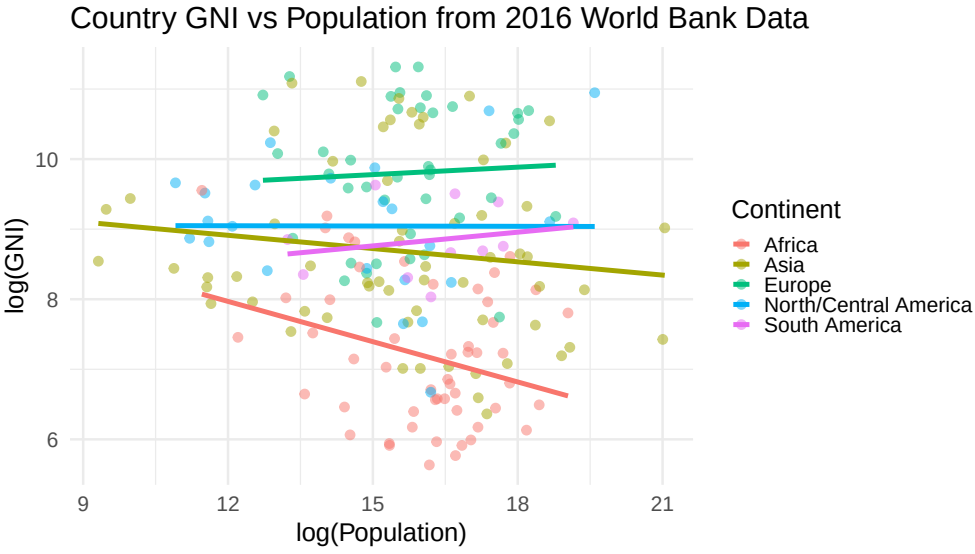
- Looks great! Now we can remake the `logPopulation` vs `logGNI` plot and color the points (and lines) by `Continent`:

Plotting 3 Variables

- Looks great! Now we can remake the `logPopulation` vs `logGNI` plot and color the points (and lines) by `Continent`:

```
WorldBank %>%
  filter(!is.na(Continent)) %>%
  ggplot(aes(x = logPopulation, y = logGNI, color = Continent)) +
  geom_point(alpha = 0.5) +
  theme_minimal(base_size = 22) +
  labs(
    title = "Country GNI vs Population from 2016 World Bank Data",
    x = "log(Population)",
    y = "log(GNI)"
  )
```

Plotting 3 Variables



Plotting 3 Variables

Plotting 3 Variables

- It looks like the relationship between `logPopulation` and `logGNI` might differ across continents! To make this even clearer, we can make 5 separate mini plots for this relationship, one for each continent:

Plotting 3 Variables

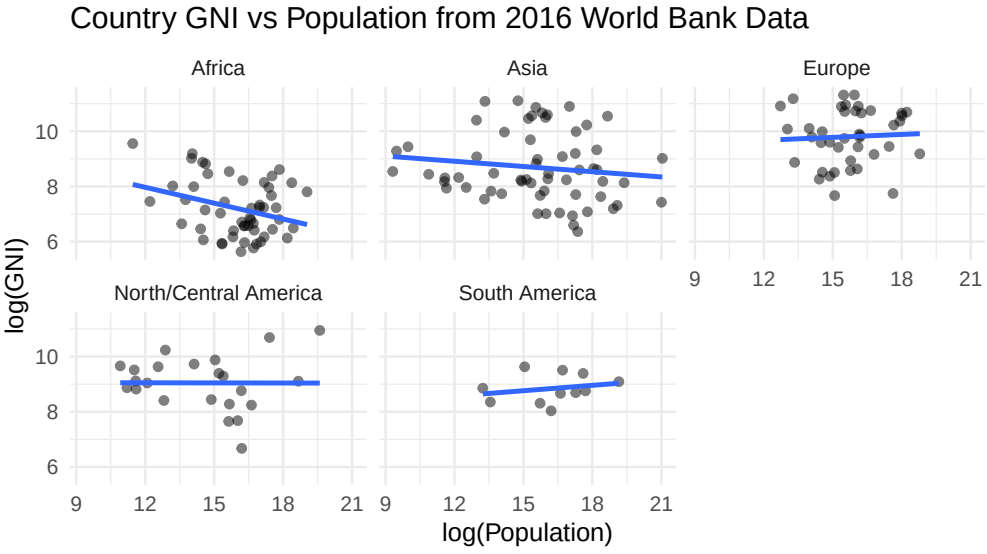
- It looks like the relationship between `logPopulation` and `logGNI` might differ across continents! To make this even clearer, we can make 5 separate mini plots for this relationship, one for each continent:

Plotting 3 Variables

- It looks like the relationship between `logPopulation` and `logGNI` might differ across continents! To make this even clearer, we can make 5 separate mini plots for this relationship, one for each continent:

```
WorldBank %>%
  filter(!is.na(Continent)) %>%
  ggplot(aes(x = logPopulation, y = logGNI)) +
  facet_wrap(~Continent) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal(base_size = 22) +
  labs(
    title = "Country GNI vs Population from 2016 World Bank Data",
    x = "log(Population)",
    y = "log(GNI)"
  )
```

Plotting 3 Variables



Saving Plots

Saving Plots

- This is a nice plot! We can save it to our `Figures` folder in our directory (make sure you have a folder named `Figures` in your working directory) by saving the `ggplot` object and using `ggsave`:

Saving Plots

- This is a nice plot! We can save it to our `Figures` folder in our directory (make sure you have a folder named `Figures` in your working directory) by saving the `ggplot` object and using `ggsave`:

Saving Plots

- This is a nice plot! We can save it to our `Figures` folder in our directory (make sure you have a folder named `Figures` in your working directory) by saving the `ggplot` object and using `ggsave`:

```
GNI_Population_by_Continent <- WorldBank %>%
  filter(!is.na(Continent)) %>%
  ggplot(aes(x = logPopulation, y = logGNI)) +
  facet_wrap(~Continent) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal(base_size = 22) +
  labs(
    title = "Country GNI vs Population from 2016 World Bank Data",
    x = "log(Population)",
    y = "log(GNI)"
  )
```

Saving Plots

```
ggsave(
  GNI_Population_by_Continent,
  filename = here("Figures", "GNI_Population_by_Continent.png"),
  width = 32,
  height = 24,
  units = "in",
  dpi = 300
)
```

Linear Regression

Theory

Definition

Ordinary least squares regression estimates the intercept and slope by minimizing the sum of the squared residuals:

$$(\hat{\beta}_0, \hat{\beta}_1) = \underset{\beta_0, \beta_1}{\arg\min} \sum_{i=1}^{n} \left[y_i - (\beta_0 + \beta_1 x_i) \right]^2 .$$

The resulting regression equation is

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i .$$

Implementation

Implementation

- You can fit linear models very easily in `R` using the `lm()` function

Implementation

- You can fit linear models very easily in `R` using the `lm()` function
- Given that it seems like the relationship between a country's GNI and its population might differ across continents, we will fit an interaction model.

Implementation

- You can fit linear models very easily in `R` using the `lm()` function
- Given that it seems like the relationship between a country's GNI and its population might differ across continents, we will fit an interaction model.
  - This will also take into account the likely main effect of `Continent` on our other two variables.

Implementation

Implementation

```
FullModel <- lm(logGNI ~ logPopulation * Continent, data = WorldBank)
summary(FullModel)
```

Implementation

```
FullModel <- lm(logGNI ~ logPopulation * Continent, data = WorldBank)
summary(FullModel)
```

- And we can further examine interactions by using the `simple_slopes` function:

Implementation

```
FullModel <- lm(logGNI ~ logPopulation * Continent, data = WorldBank)
summary(FullModel)
```

- And we can further examine interactions by using the `simple_slopes` function:

Implementation

```
FullModel <- lm(logGNI ~ logPopulation * Continent, data = WorldBank)
summary(FullModel)
```

- And we can further examine interactions by using the `simple_slopes` function:

```
library(reghelper)
simple_slopes(FullModel, pred = "logPopulation", modx = "Continent")
```

Interpretation

- From this output, we can make a few interpretations

Interpretation

- From this output, we can make a few interpretations
  - Even after controlling for the main effect of `Continent`, there is still a significant negative relationship between a country's population and its GNI.

Interpretation

- From this output, we can make a few interpretations
  - Even after controlling for the main effect of `Continent`, there is still a significant negative relationship between a country's population and its GNI.
  - When Africa is the reference group, the slope of the relationship between a country's population and its GNI is -0.19

Interpretation

- From this output, we can make a few interpretations
  - Even after controlling for the main effect of `Continent`, there is still a significant negative relationship between a country's population and its GNI.
  - When Africa is the reference group, the slope of the relationship between a country's population and its GNI is -0.19
  - No interactions are significant: the slope does not significantly change from this reference group for any of the other continents.

Interpretation

- From this output, we can make a few interpretations
  - Even after controlling for the main effect of `Continent`, there is still a significant negative relationship between a country's population and its GNI.
  - When Africa is the reference group, the slope of the relationship between a country's population and its GNI is -0.19
  - No interactions are significant: the slope does not significantly change from this reference group for any of the other continents.
  - However, when examining the slopes for different continents, the association between a country's population and its GNI is only significant for Africa.

One Last Fun Plot

One Last Fun Plot

- You can add a lot of different variables to `ggplot` object, but at risk of making the plots hard to interpret (there are only so many ways to manipulate a visual object!).

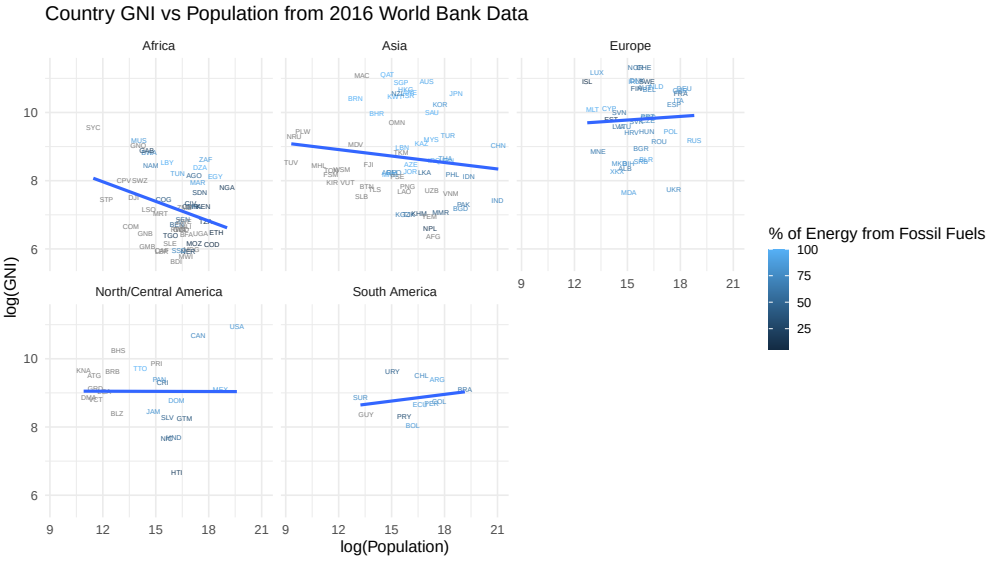
One Last Fun Plot

- You can add a lot of different variables to `ggplot` object, but at risk of making the plots hard to interpret (there are only so many ways to manipulate a visual object!).
- Let's try to add a bit for information to our plot by changing the points to the 3-character country code and by changing the color of the points based on how much fossil fuels are used in the country (as a percentage of total energy use).

Plotting 4 Variables

```
WorldBank %>%
  filter(!is.na(Continent)) %>%
  ggplot(aes(x = logPopulation, y = logGNI, color = FossilPct)) +
  geom_text(aes(label = Code), size = 2) +
  facet_wrap(~Continent) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal(base_size = 14) +
  labs(
    title = "Country GNI vs Population from 2016 World Bank Data",
    x = "log(Population)",
    y = "log(GNI)",
    color = "% of Energy from Fossil Fuels"
  )
```

Plotting 4 Variables



Plotting 4 Variables

Plotting 4 Variables

- And we can save as normal:

Plotting 4 Variables

- And we can save as normal:

Plotting 4 Variables

- And we can save as normal:

```
Pop_GNI_Fossil <- WorldBank %>%
  filter(!is.na(Continent)) %>%
  ggplot(aes(x = logPopulation, y = logGNI, color = FossilPct)) +
  geom_text(aes(label = Code), size = 2) +
  facet_wrap(~Continent) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal(base_size = 14) +
  labs(
    title = "Country GNI vs Population from 2016 World Bank Data",
    x = "log(Population)",
    y = "log(GNI)",
    color = "% of Energy from Fossil Fuels"
  )
```

Plotting 4 Variables

```
ggsave(
  Pop_GNI_Fossil,
  filename = here("Figures", "Pop_GNI_Fossil.png"),
  width = 32,
  height = 24,
  units = "in",
  dpi = 300
)
```

Summary of Exercises

Summary of Exercises

Summary of Exercises

- In this workshop, we learned how to:

Summary of Exercises

- In this workshop, we learned how to:
  - Get started with R and RStudio

Summary of Exercises

- In this workshop, we learned how to:
  - Get started with R and RStudio
  - Use R Markdown to create reproducible documents

Summary of Exercises

- In this workshop, we learned how to:
  - Get started with R and RStudio
  - Use R Markdown to create reproducible documents
  - Load in data and explore it using summary statistics and visualizations

Summary of Exercises

- In this workshop, we learned how to:
  - Get started with R and RStudio
  - Use R Markdown to create reproducible documents
  - Load in data and explore it using summary statistics and visualizations
  - Create new variables and transform existing ones

Summary of Exercises

- In this workshop, we learned how to:
  - Get started with R and RStudio
  - Use R Markdown to create reproducible documents
  - Load in data and explore it using summary statistics and visualizations
  - Create new variables and transform existing ones
  - Combine dataframes

Summary of Exercises

- In this workshop, we learned how to:
  - Get started with R and RStudio
  - Use R Markdown to create reproducible documents
  - Load in data and explore it using summary statistics and visualizations
  - Create new variables and transform existing ones
  - Combine dataframes
  - Fit linear regression models and interpret the results

Summary of Exercises

- In this workshop, we learned how to:
  - Get started with R and RStudio
  - Use R Markdown to create reproducible documents
  - Load in data and explore it using summary statistics and visualizations
  - Create new variables and transform existing ones
  - Combine dataframes
  - Fit linear regression models and interpret the results
  - Save plots for future use